

To-Do List Web Application Using HTML, CSS, and JavaScript

1. Project Title

To-Do List Web Application Using HTML, CSS, and JavaScript

2. Objective

The objective of this project is to develop a simple and interactive To-Do List application that allows users to add, complete, and delete tasks. The application helps users organize their daily activities efficiently while demonstrating the use of JavaScript for dynamic web development.

3. Introduction

A To-Do List application is a task management tool that helps users keep track of their activities and responsibilities. This project is developed using HTML, CSS, and JavaScript and provides a user-friendly interface for managing tasks.

The application stores tasks in the browser's Local Storage, ensuring that tasks remain available even after the page is refreshed or reopened.

4. Technologies Used

- HTML5
- CSS3
- JavaScript (ES6)
- Browser Local Storage

5. Project Structure

A. HTML Structure

The HTML file creates the basic layout of the application, including:

- Main container
- Heading section
- Input field
- Add button
- Task list area

B. CSS Styling

CSS is used to improve the visual appearance of the application:

- Responsive centered layout
- Modern card design
- Rounded corners
- Shadow effects

TO-DO LIST

- Colored action buttons
- Task completion styling

C. JavaScript Functionality

JavaScript provides the application's interactive features:

- Add new tasks
- Mark tasks as completed
- Delete tasks
- Save tasks using Local Storage
- Display saved tasks automatically

6. Features of the Application

1. Add Task

Users can enter a task in the input field and click the "Add" button to include it in the task list.

2. Complete Task

Tasks can be marked as completed using the checkmark button. Completed tasks appear with:

- Strikethrough text
- Gray color

3. Delete Task

Users can remove tasks permanently using the delete button.

4. Local Storage Support

All tasks are stored in the browser's Local Storage, allowing data persistence even after refreshing the page.

5. Dynamic User Interface

The task list updates instantly without reloading the webpage.

7. Working of the Program

Step 1: Load Existing Tasks

When the webpage loads, tasks are retrieved from Local Storage.

Step 2: Add New Task

TO-DO LIST

The user enters a task and clicks the Add button. The task is stored in the tasks array and Local Storage.

Step 3: Display Tasks

The `renderTasks()` function dynamically creates task items and displays them on the screen.

Step 4: Complete Task

The `toggleTask()` function changes the completion status of a task.

Step 5: Delete Task

The `deleteTask()` function removes a task from the array and updates Local Storage.

8. Functions Used

saveTasks()

Stores all tasks in Local Storage.

renderTasks()

Displays all tasks on the webpage.

addTask()

Adds a new task to the task list.

toggleTask(index)

Marks a task as completed or incomplete.

deleteTask(index)

Deletes a selected task from the list.

9. User Interface Description

Input Section

- Text input field
- Add button

Task Section

- Displays all tasks
- Complete button (✓)
- Delete button (✕)

TO-DO LIST

Styling Features

- Clean design
- Responsive layout
- Shadow effects
- Color-coded action buttons

10. Advantages

- Easy to use
- Fast task management
- Data saved automatically
- Lightweight application
- No internet connection required
- Beginner-friendly project

11. Limitations

- No task editing feature
- No task categories
- No due date management
- Works only within the same browser

12. Future Enhancements

- Edit existing tasks
- Add task priorities
- Add due dates and reminders
- Dark mode support
- Task search and filtering
- Cloud database integration
- User authentication system

13. Output

The application displays:

- Input field for task entry
- Add button
- Dynamic list of tasks
- Complete and Delete options for each task

14. Conclusion

The To-Do List Web Application successfully demonstrates the use of HTML, CSS, and JavaScript to create an interactive and responsive web application. The integration of Local Storage ensures persistent data storage, while the simple user interface makes task management easy and efficient. This project provides a strong foundation for learning JavaScript DOM manipulation and client-side storage techniques.

TO-DO LIST

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>To-Do App</title>
<style>
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial, sans-serif;
}
body{
background:#f4f4f4;
display:flex;
justify-content:center;
align-items:center;
min-height:100vh;
}
.container{
width:400px;
background:#fff;
padding:20px;
border-radius:10px;
box-shadow:0 5px 15px rgba(0,0,0,0.2);
}
h2{
text-align:center;
margin-bottom:20px;
}
.input-box{
display:flex;
gap:10px;
}
input{
flex:1;
padding:10px;
```

TO-DO LIST

```
border:1px solid #ccc;
border-radius:5px;
}
button{
padding:10px 15px;
border:none;
background:#007bff;
color:#fff;
border-radius:5px;
cursor:pointer;
}
ul{
list-style:none;
margin-top:20px;
}
li{
display:flex;
justify-content:space-between;
align-items:center;
padding:10px;
margin-bottom:10px;
background:#eee;
border-radius:5px;
}
li.completed span{
text-decoration:line-through;
color:gray;
}
.actions button{
margin-left:5px;
padding:5px 8px;
}
.delete{
background:red;
}
.complete{
background:green;
}
</style>
</head>
```

TO-DO LIST

```
<body>
<div class="container">
<h2>To-Do List</h2>
<div class="input-box">
<input type="text" id="taskInput" placeholder="Enter task">
<button onclick="addTask()">Add</button>
</div>

<ul id="taskList"></ul>
</div>

<script>
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
function saveTasks(){
localStorage.setItem("tasks", JSON.stringify(tasks));
}
function renderTasks(){
const taskList = document.getElementById("taskList");
taskList.innerHTML = "";
tasks.forEach((task, index) => {
const li = document.createElement("li");
if(task.completed){
li.classList.add("completed");
}
li.innerHTML = `
<span>${task.text}</span>
<div class="actions">
<button class="complete"
onclick="toggleTask(${index})">
✓
</button>
<button class="delete"
onclick="deleteTask(${index})">
✕
</button>
</div>
`;
taskList.appendChild(li);
});
}
function addTask(){
```

TO-DO LIST

```
const input = document.getElementById("taskInput");
const text = input.value.trim();
if(text === "") return;
tasks.push({
  text: text,
  completed: false
});
saveTasks();
renderTasks();
input.value = "";
}

function toggleTask(index){
  tasks[index].completed =
    !tasks[index].completed;
  saveTasks();
  renderTasks();
}

function deleteTask(index){
  tasks.splice(index, 1);
  saveTasks();
  renderTasks();
}

renderTasks();
</script>
</body>
</html>
```

OUTPUT:

